Apache Kafka

Organisatorisches

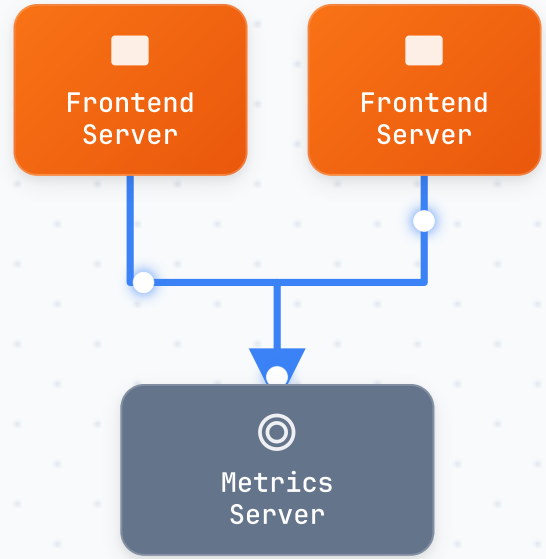
- Podcast: **All About Kafka**
- Mara + Manuel

Wieso lohnt es sich zuzuhören?

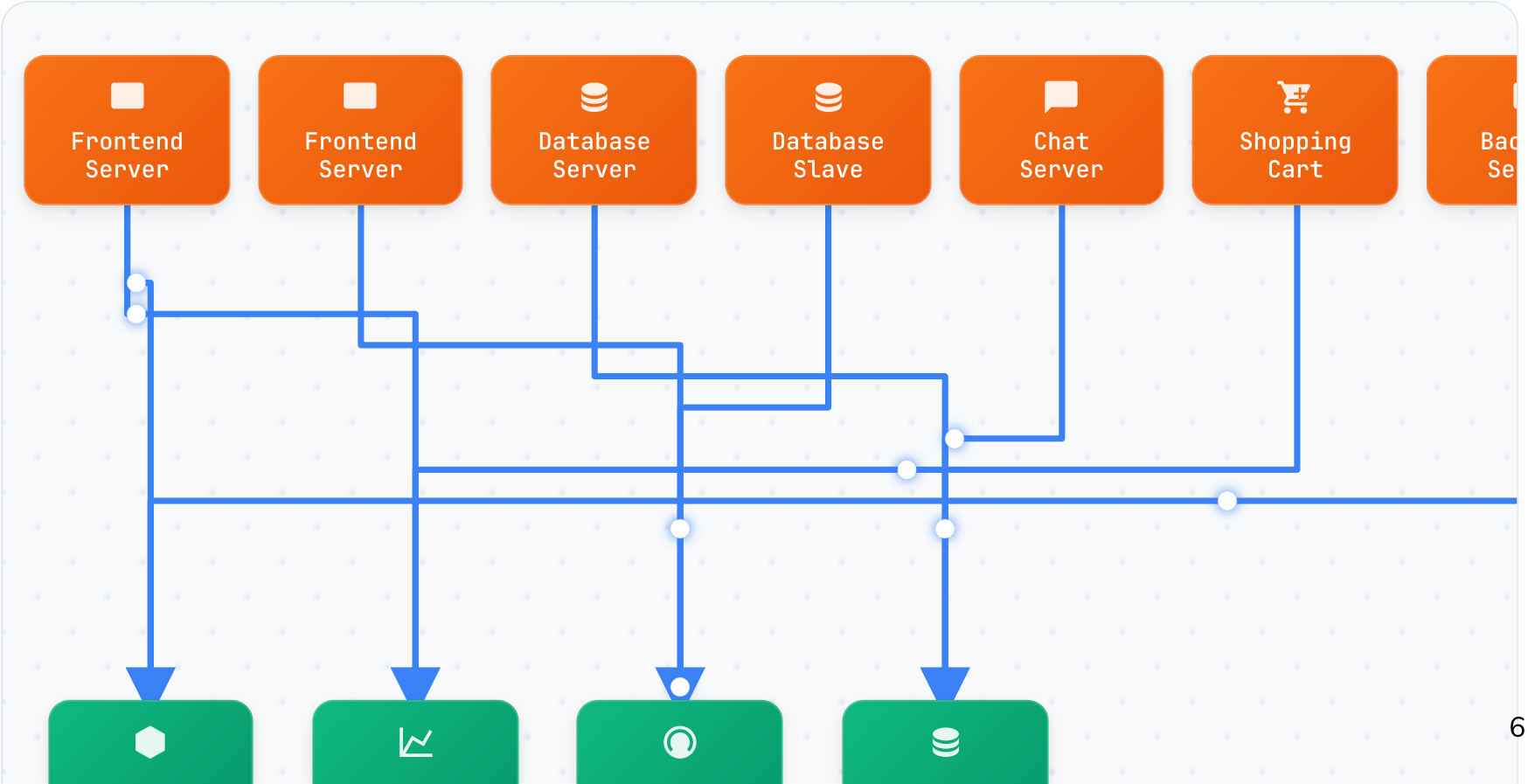
80 % der Fortune-100-Unternehmen nutzen Kafka

Publish / Subscribe

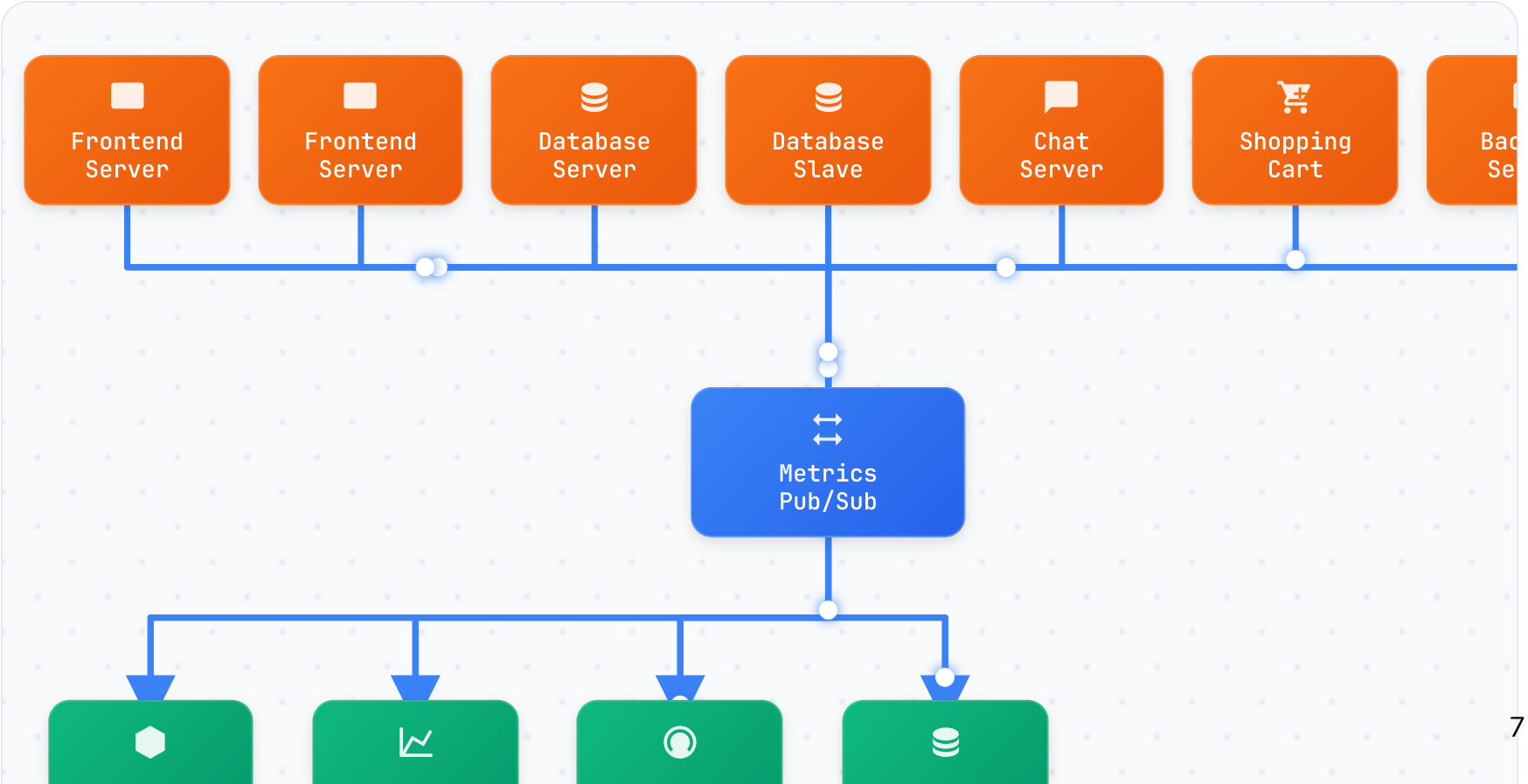
Publish/Subscribe



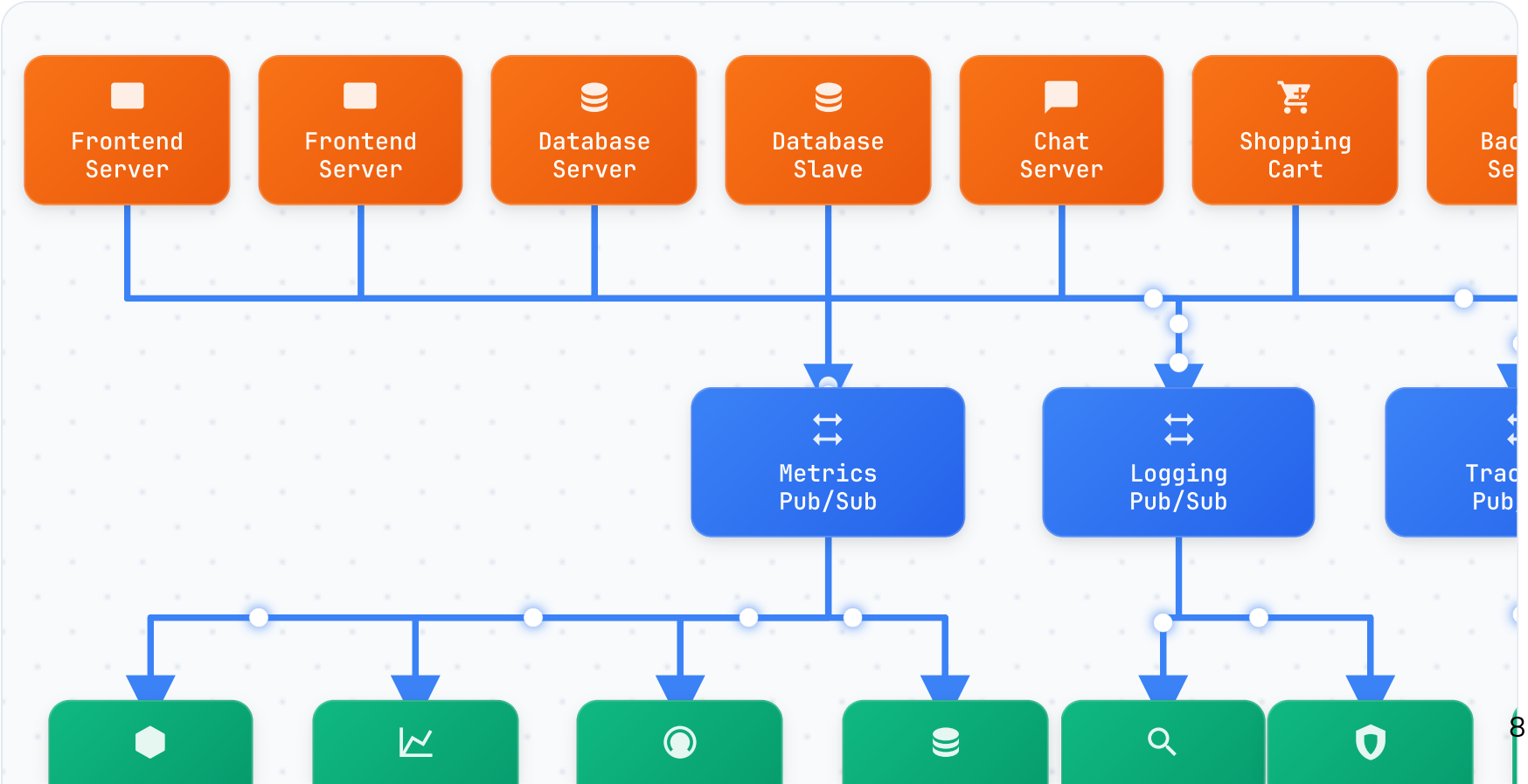
Publish/Subscribe



Publish/Subscribe



Publish/Subscribe



Kafka

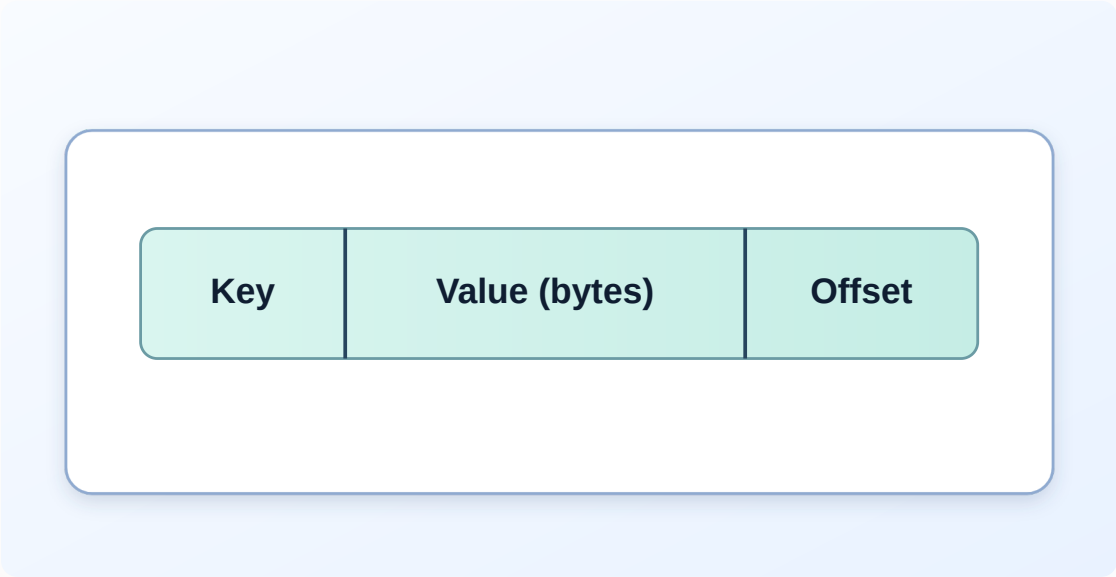
- Publish/Subscribe-Messaging-System
- Dauerhafte Speicherung
- Deterministisch lesbar
- Datenverteilung über mehrere Systeme

Producer

- Producer erzeugen Messages und senden sie an Kafka

Messages

- Dateineinheit = **Message**
- Message und Key sind Byte-Arrays
- Offset identifiziert die Position in der Partition



Schemas

- Konsistentes Datenformat ermöglicht stabile Kommunikation
- Versionierung verhindert Breaking Changes bei Producer/Consumer
- Schemas liegen oft in einer zentralen Registry

Schema-Formate

JSON

Einfach lesbar, weit verbreitet

XML

Streng strukturiert, verbose

Apache Avro

Kompakt, schema-basiert, performant

Partition

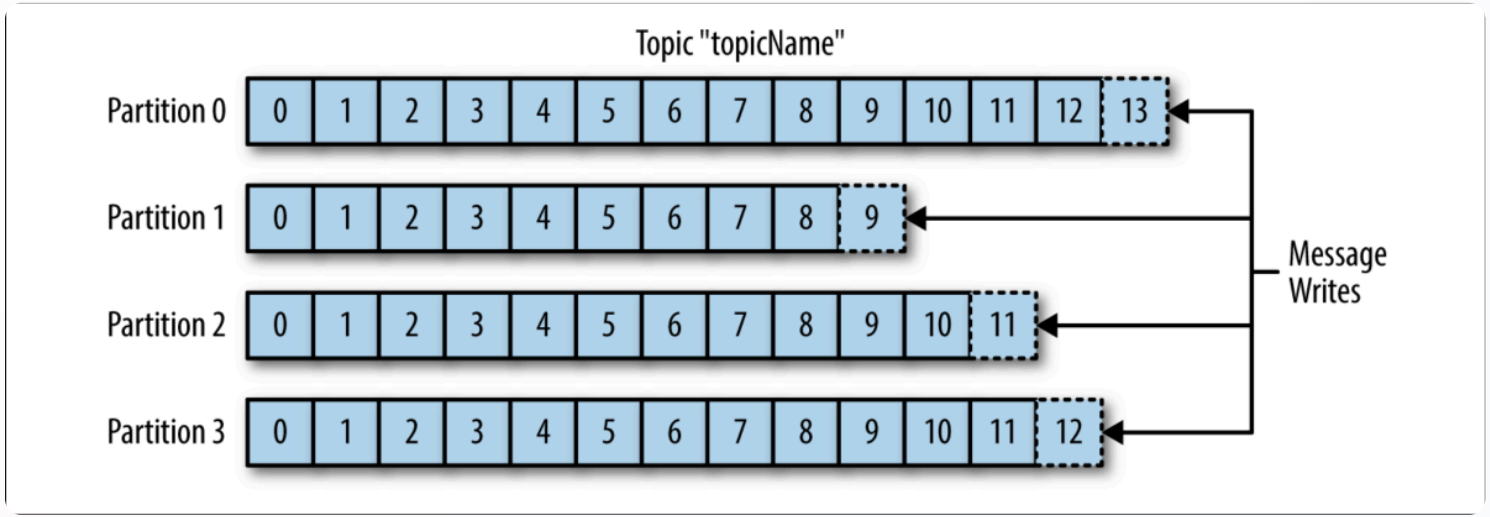
- Kleinste Speichereinheit in Kafka
- Append-only
- Message Key bestimmt Zielpartition
- Reihenfolge gilt strikt pro Partition

Batches

- Menge an Messages pro Topic und Partition
- Verbessert Speicher- und Netzwerkeffizienz
- Trade-off zwischen Latenz und Throughput

Topic

- Topic vergleichbar mit Tabelle
- Topic kann mehrere Partitionen haben
- Partitionen können über mehrere Kafka Server verteilt sein



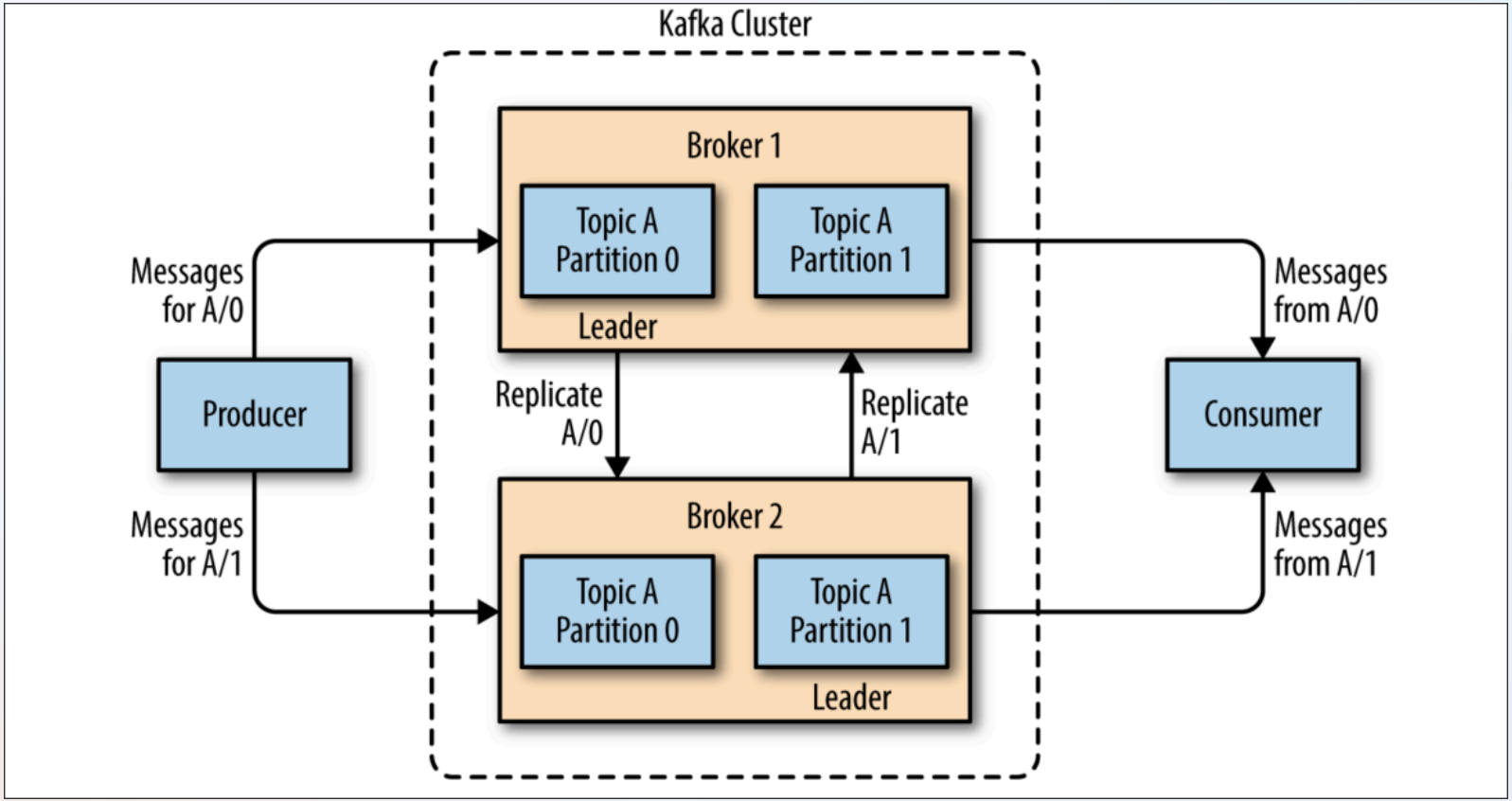
Brokers

- Broker = ein Kafka-Server
- Nimmt Messages an, vergibt Offsets und speichert sie
- Liefert Messages an Consumer aus
- Verarbeitet viele Partitionen mit hohem Durchsatz

Broker Clusters

- Broker sind für Clusterbetrieb ausgelegt
- Ein Broker agiert als Controller
- Controller überwacht Broker-Ausfälle und Partition-Zuteilung
- Jede Partition hat einen Leader-Broker

Cluster-Architektur



Replikation & Failover

- Partitionen können auf mehrere Broker repliziert werden
- **Leader:** Verarbeitet alle Schreib- und Lesezugriffe
- **Follower:** Kopieren Daten passiv vom Leader (Redundanz)
- **Failover:** Automatischer Leader-Wechsel bei Ausfall
- Producers/Consumers müssen sich zum Leader verbinden

Disk-basierte Retention

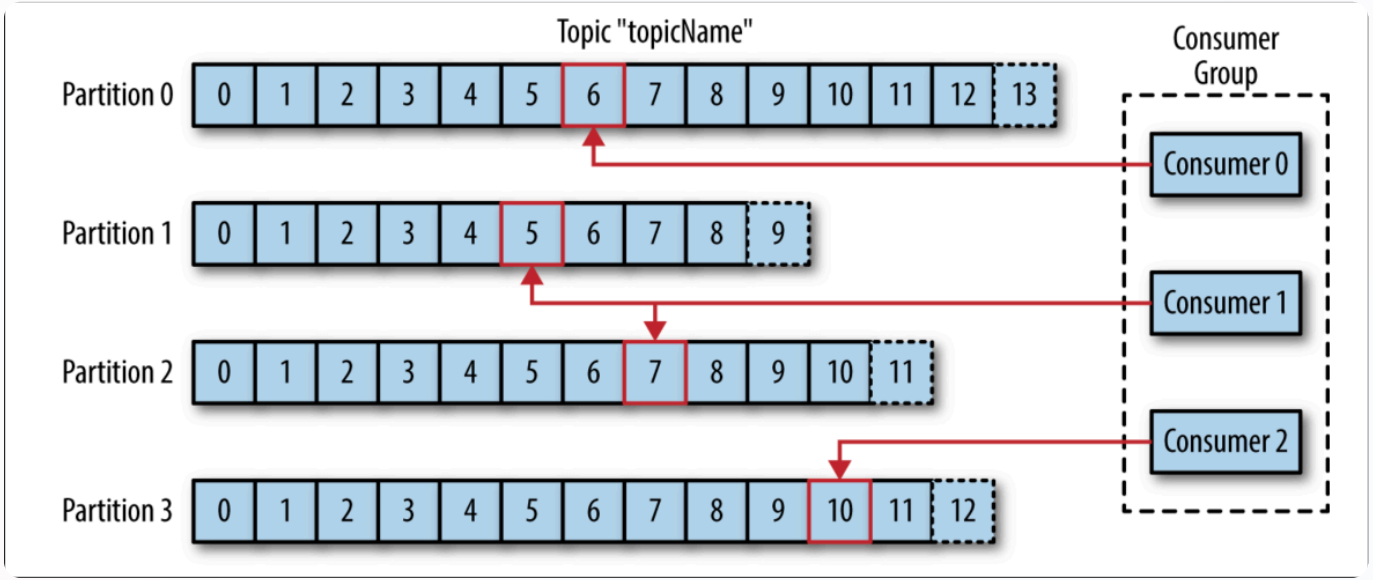
- Messages werden dauerhaft auf Disk gespeichert
- Konfigurierbare Regeln pro Topic (Zeit oder Größe)
- Consumer müssen nicht in Echtzeit arbeiten
- Schutz vor Datenverlust bei Lastspitzen oder Wartung

Consumers

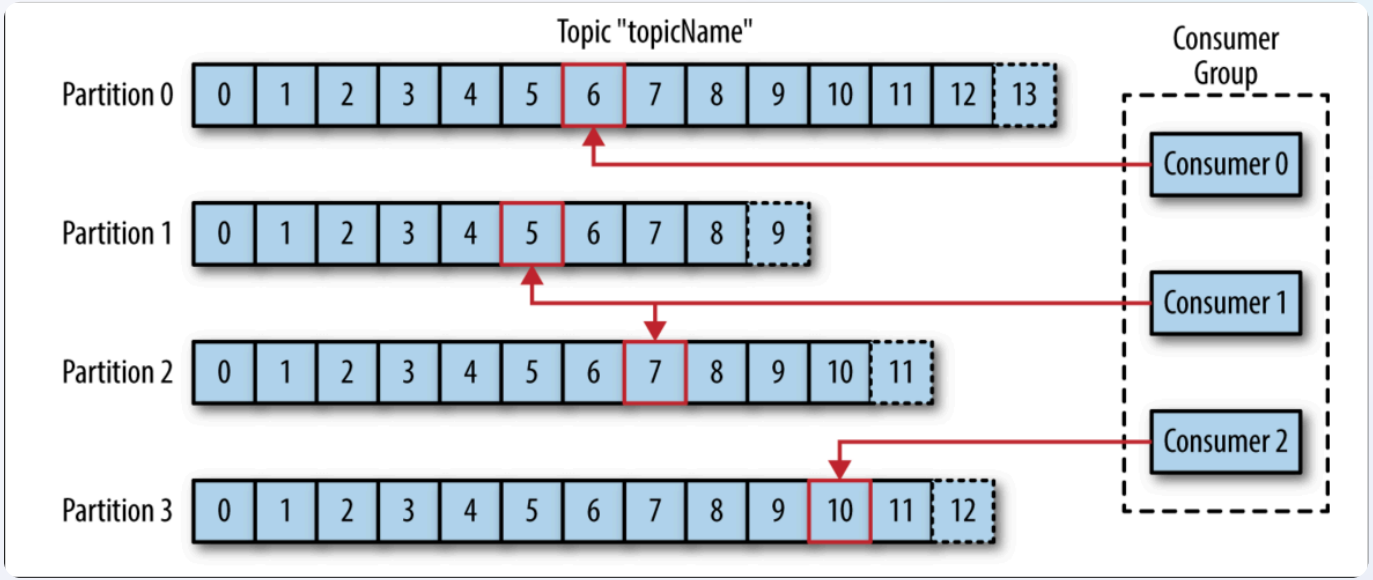
- Lesen Nachrichten in Erstellungsreihenfolge pro Partition
- Speichern Offsets für idempotente Verarbeitung
- Können jederzeit neu starten und fortsetzen

Consumer Group

- Mehrere Consumer arbeiten als Gruppe
- Eine Partition wird nur von einem Consumer der Gruppe gelesen
- Skalierung erfolgt über mehr Consumer und Partitionen



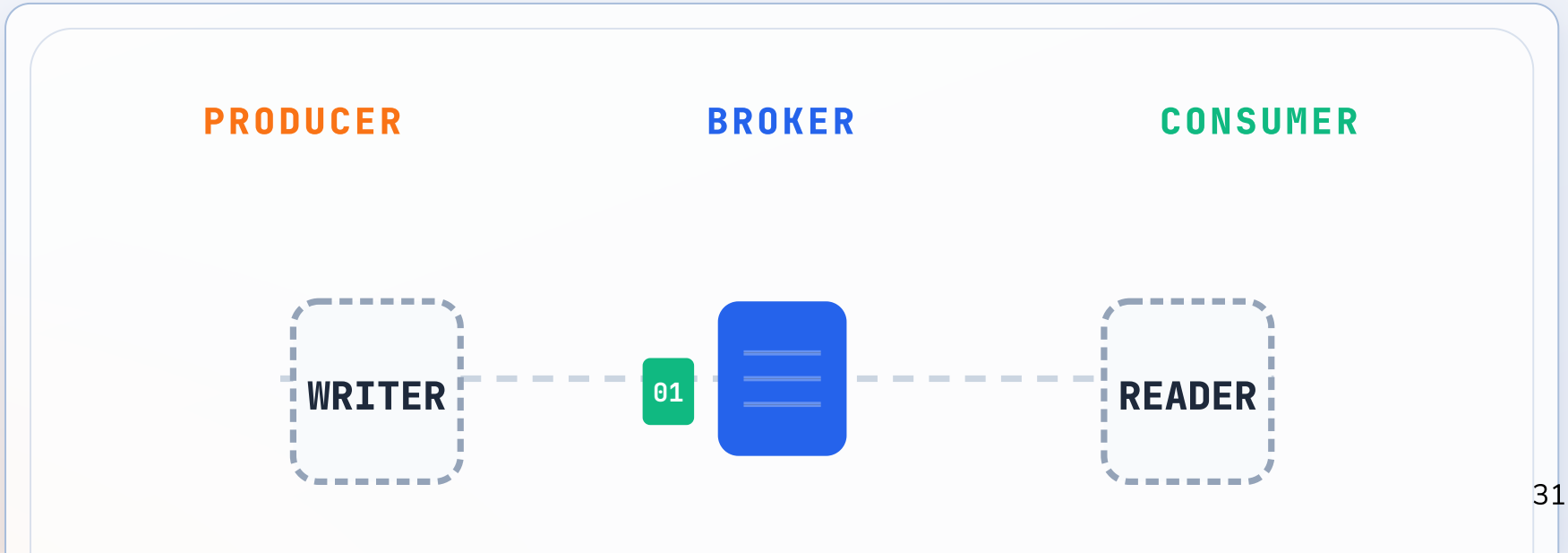
Consumer Group



Sources

1. Apache Kafka "Powered By" page: https://kafka.apache.org/powered-by/
2. Kafka: The Definitive Guide (Neha Narkhede, Gwen Shapira, Todd Palino, O’Reilly, 2017), ISBN 978-1-491-93616-0

Vielen Dank für eure Aufmerksamkeit!

Apache Avro

- JSON-orientiert, aber effizient für Big-Data-Workloads
- Besteht aus Schema + kompakter Binary Data
- Writer: Schema + Data → Binary
- Reader: Schema + Binary → Data